

Esercizio 1

Object Detection con YOLO

Guida di studio: teoria, dati, modello, valutazione e domande d'esame

In una riga: un detector in stile **YOLO** scritto a mano (griglia, anchor, loss, IoU, NMS) sopra una **ResNet18 pre-addestrata**, addestrato su un sottoinsieme di **Object365** (14.361 immagini, 8 classi di oggetti medio-grandi) per riconoscere oggetti in una foto.

Indice

1. Il quadro in una pagina	3
2. La consegna e il dataset Object365	3
3. Le scelte sui dati	4
4. Richiami sulle CNN	6
5. Dalla localizzazione a YOLO	7
6. YOLO nel progetto	8
7. L'architettura	10
8. La loss	11
9. Inferenza: IoU e non-max suppression	12
10. Come si misura un detector	14
11. Il training	15
12. Gli esperimenti di ottimizzazione	16
13. Risultati	17
14. Come è organizzato il codice	21
15. Limiti e possibili miglioramenti	22
16. Domande d'esame probabili	22
17. Glossario	24

1. Il quadro in una pagina

Questa guida spiega l'esercizio 1 dall'inizio alla fine: la consegna, le scelte sui dati, la teoria della detection con YOLO, come è costruita e addestrata la rete e come si misurano i risultati. I numeri sui dati e sul modello sono calcolati sul progetto reale.

Aspetto	Scelta nel progetto
Compito	Object detection: trovare gli oggetti in una foto e disegnare una bounding box con la classe
Dataset	Object365, sottoinsieme di 14.361 immagini e 119.215 box
Classi	Person, Chair, Table, Cabinet, Car, Lamp, Picture, Monitor (oggetti medio-grandi)
Tecnica	YOLO (griglia + anchor), implementato a mano: target, loss, decodifica, IoU, NMS
Rete	ResNet18 pre-addestrata su ImageNet (troncata) + collo + testa 1×1; 1,59 M parametri a stride 8
Valutazione	mAP@0.5 sul validation per confrontare le varianti; test set usato una sola volta alla fine
Ottimizzazione	8 esperimenti controllati: ognuno cambia una sola cosa rispetto alla baseline
Risultato	mAP@0.5 = 0,444 sul test (baseline 0,221), F1 0,588, precision 0,66, recall 0,53

Le 6 idee da portare all'esame

- Il risultato dipende più dalle **scelte di progetto** che dal modello: stride e augmentation, misurati con esperimenti controllati, hanno raddoppiato il mAP.
- **YOLO** divide l'immagine in una griglia: ogni oggetto è di competenza della cella che contiene il suo centro, e la rete predice tutte le box in un solo passaggio (*You Only Look Once*).
- Gli **anchor** sono forme di partenza: ogni cella ha più anchor, e un oggetto viene assegnato a quello con la forma più simile (IoU calcolata solo su larghezza e altezza).
- La rete non predice coordinate assolute ma **correzioni**: $\sigma(t)$ sposta il centro dentro la cella, e^t allarga o stringe l'anchor.
- La **loss** somma quattro termini (coordinate, objectness, no-object, classe); il peso del no-object evita sia la rete che non vede mai nulla sia quella che vede oggetti ovunque.
- L'**IoU** misura quanto due box si sovrappongono; serve per la **NMS** (eliminare i duplicati) e per decidere se una predizione è corretta.
- La **precision** dipende dalla soglia di confidenza: per confrontare modelli si usa l'**AP**, l'area sotto la curva precision-recall, mediata sulle classi (**mAP**).

2. La consegna e il dataset Object365

2.1 Cosa chiede l'esercizio

La traccia chiede di *applicare una delle tecniche viste a lezione per l'object detection tramite bounding box*, per costruire un riconoscitore di oggetti in un'immagine presa con una fotocamera, usando **Object365** ed eseguendo un **subsampling** se necessario. Due parole chiave guidano le scelte: “tecnica vista a lezione” (quindi YOLO con anchor, IoU e NMS, implementati e non importati da una libreria) e “subsampling” (il dataset completo è ingestibile).

2.2 Object365

Object365 contiene circa 600.000 immagini ad alta qualità con 365 categorie e circa 10 milioni di box. Il progetto usa la copia su HuggingFace [surenreddy/object365](https://huggingface.co/surenreddy/object365), divisa in file parquet da circa 120 MB (35 file per ogni blocco): ogni riga contiene l'immagine JPEG e le sue annotazioni. Ogni annotazione ha la box in pixel (angolo in alto a sinistra, larghezza, altezza), l'id della categoria e il flag **iscrowd**. Se ne scaricano **15 file** (circa 1,9 GB).

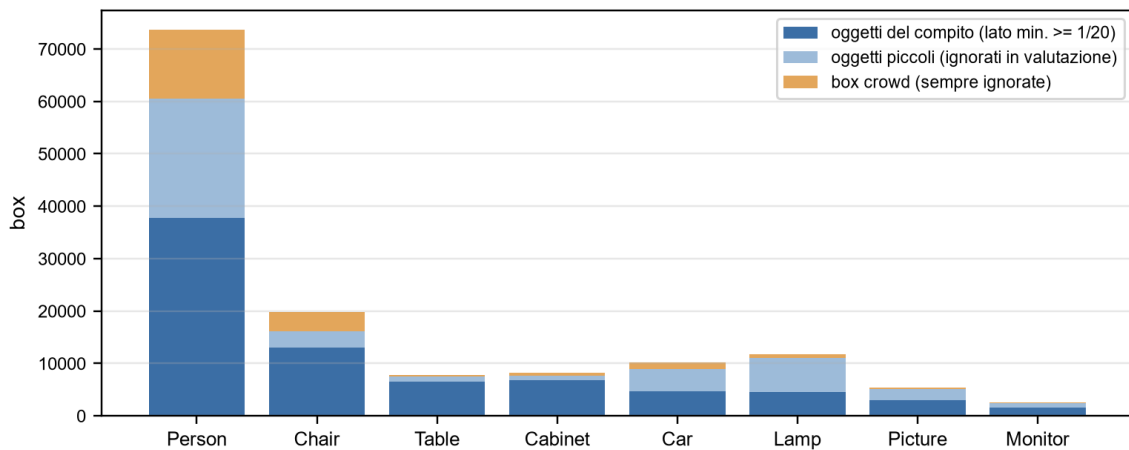
3. Le scelte sui dati

3.1 Otto classi scelte sui dati

Le 8 classi non sono scelte a intuito. Si sono misurate, per tutte le categorie presenti nei file scaricati, frequenza e dimensione tipica delle box, tenendo quelle **frequenti e di dimensione medio-grande**. Una prima versione del progetto usava Bottle, Cup, Hat e Book: le prime tre hanno lato minore mediano intorno al 3% del lato dell'immagine (troppo piccole per il detector), e Book era per due terzi fatto di box crowd (intere librerie).

Classe	Categorie Object365 (id)	Box	Box crowd
Person	Person (1)	60.535	13.132
Chair	Chair (3), Stool (48)	16.099	3.676
Table	Desk (10), Side Table (169)	7.424	414
Cabinet	Cabinet/shelf (13)	7.616	644
Car	Car (6), SUV (35), Van (50), Pickup Truck (88)	8.973	1.247
Lamp	Lamp (7)	11.000	725
Picture	Picture/Frame (17)	5.126	211
Monitor	Monitor/TV (38)	2.442	49

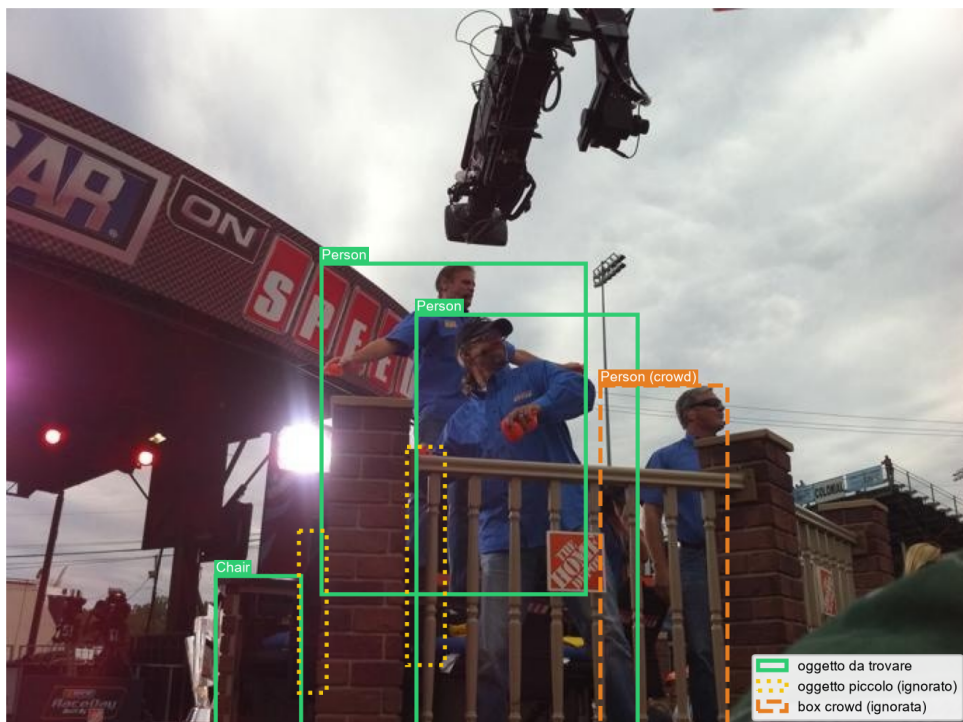
- **Categorie sorelle unite:** auto, SUV, furgoni e pick-up sono quasi indistinguibili. Tenendo solo "Car", un SUV nella foto sarebbe etichettato come sfondo mentre un'auto identica accanto è un oggetto: la rete riceverebbe segnali contraddittori. Lo stesso vale per scrivanie e tavolini, sedie e sgabelli.
- **Id verificati a occhio:** gli id numerici sono stati controllati ritagliando esempi dalle immagini. Dining Table (99) e Coffee Table (168) sono stati esclusi perché gli esempi erano assenti o sbagliati. I ritagli mostrano anche un certo **rumore nelle etichette** (circa un ritaglio su dieci palesemente errato).
- **Sbilanciamento:** Person da sola è il 51% delle box; Monitor meno del 2%. Ci si aspettano prestazioni più basse sulle classi rare.



Box per classe: oggetti del compito, oggetti piccoli e box crowd (tutte ignorate in valutazione tranne le prime).

3.2 Box crowd e oggetti piccoli: le regioni ignorate

In Object365 una box **crowd** racchiude un gruppo di oggetti che non sono stati annotati uno per uno (una folla, una fila di sedie). Nei dati sono 20.098 su 139.313. Chiedere alla rete di predire una sola box attorno a un gruppo è un obiettivo sbagliato; cancellarle sarebbe ugualmente sbagliato, perché le persone restano nella foto e la rete imparerebbe che “lì non c'è nulla”. La soluzione, la stessa di COCO, è trattarle come **regioni ignorate**: non generano obiettivi e non producono penalità.



Un'immagine del validation set: in verde gli oggetti da trovare, in arancione una box crowd (un gruppo di persone dietro la ringhiera), in giallo due oggetti troppo piccoli. Le ultime due categorie sono ignorate.

Lo stesso trattamento vale per gli **oggetti piccoli**. Un detector a scala singola non può localizzare un oggetto più piccolo di una cella della griglia. Quanti oggetti finiscono sotto questo limite dipende dalla griglia:

Griglia	Lato di una cella	Oggetti più piccoli di una cella
52×52	1/52 dell'immagine	8,8%
26×26	1/26 dell'immagine	25,8%
20×20	1/20 dell'immagine	34,7%
16×16	1/16 dell'immagine	42,9%
13×13	1/13 dell'immagine	50,9%

La definizione del compito

Il compito valutato è: **trovare gli oggetti con lato minore almeno 1/20 del lato dell'immagine** (77.818 oggetti). La soglia è fissa e non dipende dalla griglia del modello: così tutte le varianti vengono confrontate sugli stessi oggetti. Se ogni variante usasse la propria soglia, una griglia più grossolana toglierebbe gli oggetti difficili e sembrerebbe migliore senza esserlo.

3.3 Formato delle etichette e split

Ogni immagine ha un file di testo con una riga per box: `classe cx cy w h crowd`. Le coordinate sono **normalizzate** in $[0, 1]$ rispetto alla dimensione dell'immagine (centro, larghezza, altezza), quindi restano valide dopo il ridimensionamento. Le immagini sono divise a caso (seme fisso) in:

Split	Immagini	Uso
train	11.467	aggiornare i pesi
validation	1.395	scegliere il checkpoint e confrontare le varianti
test	1.499	valutazione finale, una sola volta

4. Richiami sulle CNN

4.1 Convoluzione, stride, pooling

Una **convoluzione** fa scorrere un piccolo filtro (per esempio 3×3) sull'immagine e a ogni posizione calcola un prodotto scalare: il risultato è una **feature map**. I pesi del filtro sono appresi. Due proprietà la rendono adatta alle immagini: **parameter sharing** (lo stesso rilevatore di bordi è utile ovunque) e **connessioni sparse** (ogni uscita dipende solo da una piccola zona dell'ingresso).

- **Stride**: di quanti pixel si sposta il filtro. Con stride 2 la feature map ha metà risoluzione.
- **Max pooling**: tiene il massimo di ogni regione; riduce la risoluzione e rende le feature più robuste a piccoli spostamenti.
- **Stride totale** della rete: il prodotto degli stride di tutti gli strati. Con stride totale 8, un'immagine 416×416 diventa una feature map 52×52 : ogni posizione corrisponde a un blocco di 8×8 pixel.

4.2 Campo recettivo

Il **campo recettivo** di un'unità è la porzione di immagine che può influenzarla. Cresce a ogni strato: un filtro $k \times k$ aggiunge $(k - 1) \times j$ pixel, dove j è lo stride accumulato fino a quel punto. È decisivo per la detection: per riconoscere un tavolo che occupa metà foto, la cella deve “vedere” una parte consistente del tavolo.

4.3 Batch normalization, ResNet e transfer learning

- **Batch normalization:** normalizza le attivazioni di ogni strato usando media e varianza del mini-batch (più scala e traslazione apprese). Stabilizza e accelera il training e ha un leggero effetto di regolarizzazione.
- **ResNet:** introduce le **connessioni residue** (l'uscita di un blocco è $x + F(x)$). Il gradiente passa direttamente attraverso la somma, rendendo addestrabili reti profonde. ResNet18 ha 4 stadi (layer1–layer4) che portano lo stride da 4 a 32.
- **Transfer learning:** si parte da una rete pre-addestrata su ImageNet (1,2 milioni di immagini, 1000 classi), i cui filtri riconoscono già bordi, texture e parti di oggetti, e la si adatta al nuovo compito (fine-tuning). Con pochi dati il vantaggio è enorme rispetto ai pesi casuali.

5. Dalla localizzazione a YOLO

5.1 Classificazione con localizzazione

Nelle slide il punto di partenza è un'immagine con un solo oggetto. La rete produce un vettore $y = [p_c, b_x, b_y, b_h, b_w, c_1, \dots, c_n]$: p_c dice se c'è un oggetto, le b descrivono la box, le c la classe. Se $p_c = 0$ il resto è “don't care” e non entra nella loss.

5.2 Sliding window e sua versione convoluzionale

Per più oggetti, l'idea ingenua è la **sliding window**: far scorrere una finestra sull'immagine, a più dimensioni, e classificare ogni ritaglio. Il problema è il **costo**: migliaia di ritagli, ognuno passato nella rete. La soluzione delle slide è implementarla **convoluzionalmente**: trasformando gli strati densi in convoluzioni, una sola passata sull'immagine intera produce le uscite di tutte le finestre insieme, condividendo i calcoli.

5.3 L'idea di YOLO

- L'immagine è divisa in una **griglia $S \times S$** ; la rete produce, per ogni cella, un vettore come quello di 5.1.
- Ogni oggetto è assegnato a **una sola cella**: quella che contiene il suo centro.
- Il centro è espresso **rispetto alla cella** (tra 0 e 1), larghezza e altezza rispetto alla cella (possono superare 1 se l'oggetto è più grande).
- Per gestire più oggetti con il centro nella stessa cella, ogni cella ha più **anchor box**: forme predefinite. Ogni oggetto va all'anchor di forma più simile. Le slide notano che numero e forma degli anchor si scelgono “a mano”.
- Tutto in **un solo passaggio** della rete: da qui “You Only Look Once” e la velocità del metodo.

6. YOLO nel progetto

6.1 Cella responsabile e anchor

Griglia 13x13 (nel modello 52x52): la cella gialla contiene il centro della persona

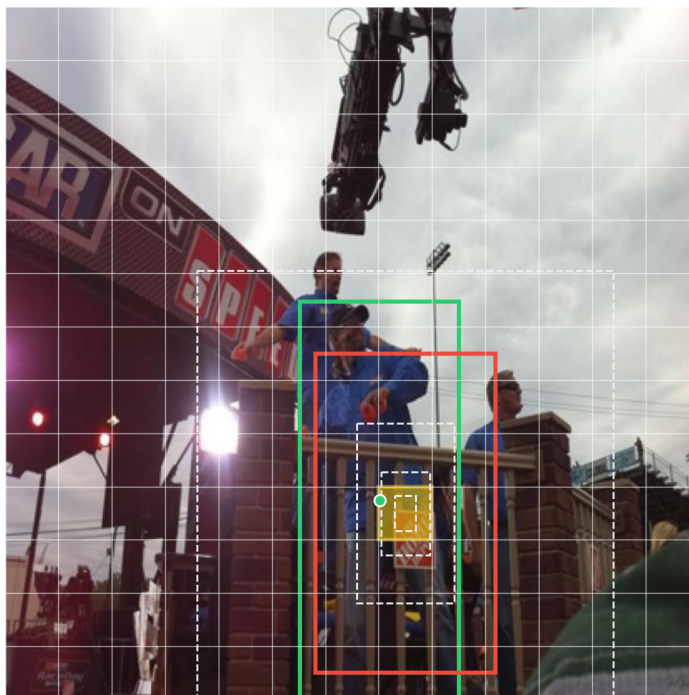
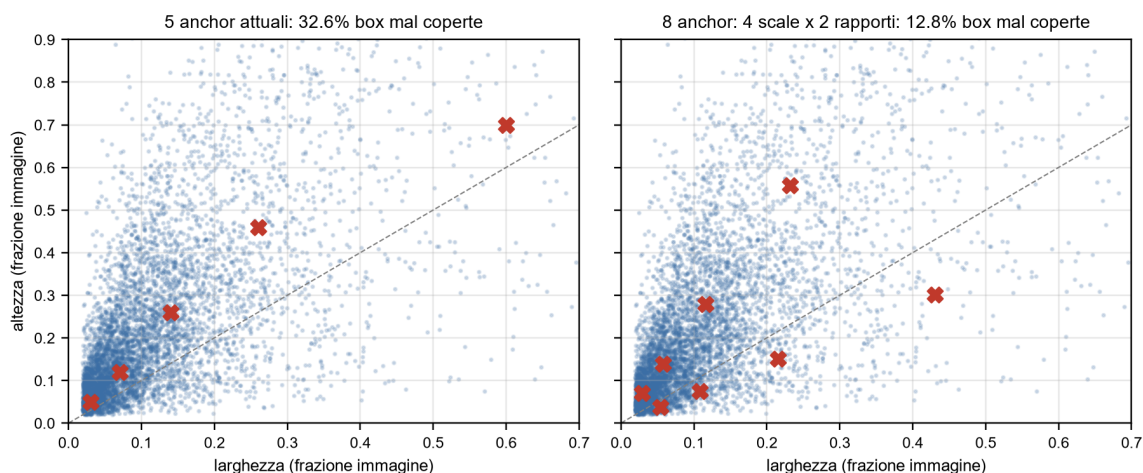


Immagine 416×416 come la vede la rete, con una griglia 13×13 per leggibilità. In verde la persona, in giallo la cella che ne contiene il centro, tratteggiati i 5 anchor centrati nella cella; in rosso quello scelto.

La persona ha larghezza 0,229 e altezza 0,574 (frazioni dell'immagine). Confrontando solo le forme, cioè immaginando box e anchor centrati nello stesso punto, le IoU con i 5 anchor valgono 0,01, 0,06, 0,28, 0,72, 0,31: vince l'anchor 4, che diventa **responsabile** dell'oggetto. Solo quell'anchor, in quella cella, riceve gli obiettivi di coordinate, objectness e classe.

6.2 Scegliere gli anchor

Nella configurazione di partenza gli anchor sono 5 forme scelte a mano. Sul dataset attuale il 33% delle box non ha nessun anchor con IoU di forma almeno 0,5: tutti gli anchor sono verticali, mentre il 12% degli oggetti è orizzontale (tavoli, auto) e un quarto è molto alto (persone in piedi). Una costruzione sistematica, sempre “a mano” ma con due regole (4 scale che raddoppiano per 2 rapporti d'aspetto), scende al 13%. È uno degli esperimenti.



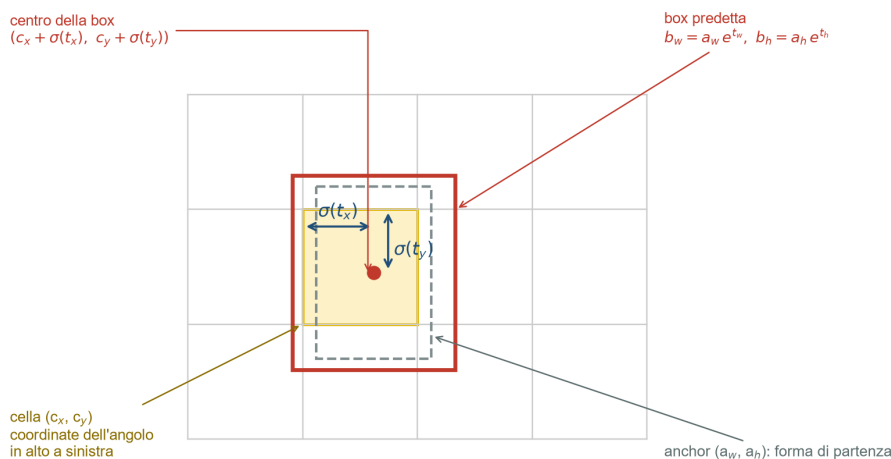
Ogni punto è una box del dataset (larghezza, altezza); le croci sono gli anchor.

6.3 Cosa predice la rete

Per ogni cella e per ogni anchor la rete produce 5 + C numeri, con C = 8 classi: t_x, t_y, t_w, t_h (la box), t_o (objectness: c'è un oggetto?) e 8 logit di classe. Con griglia 52×52 e 5 anchor sono 13.520 box candidate per immagine. L'uscita ha forma (5, 52, 52, 13).

6.4 Decodifica delle coordinate

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = a_w e^{t_w}, \quad b_h = a_h e^{t_h}$$



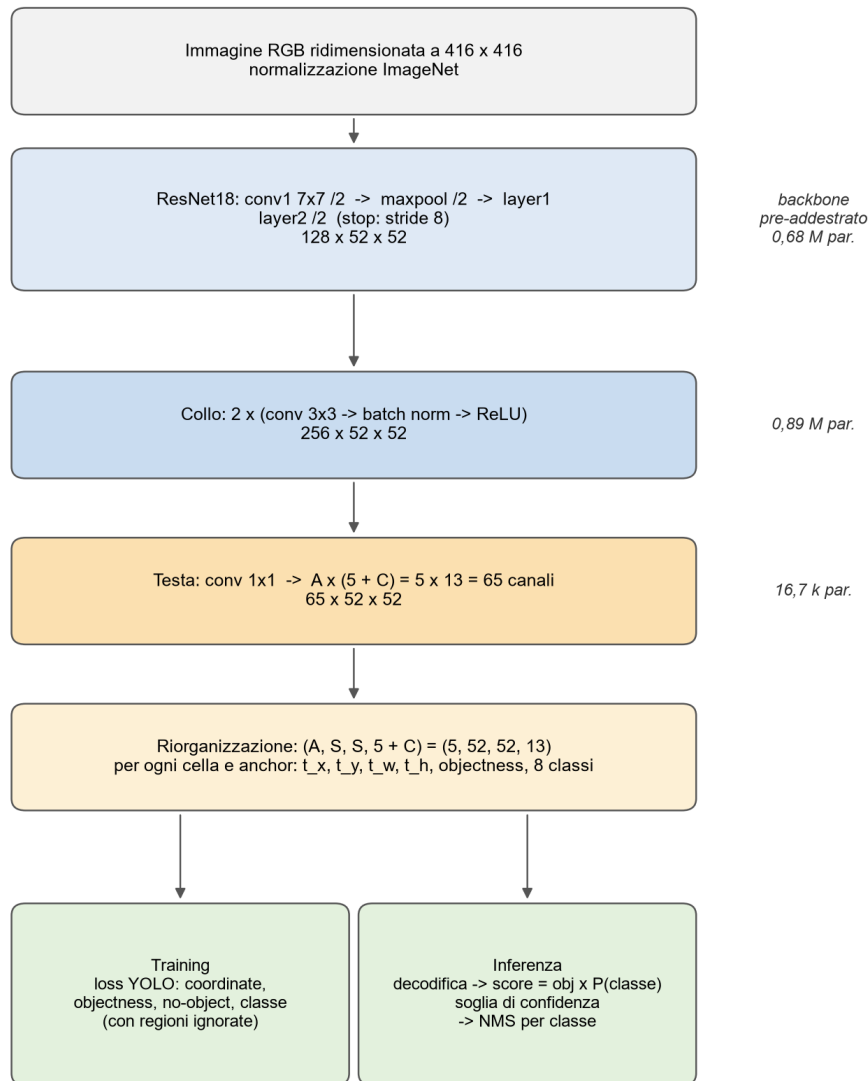
Come i numeri grezzi diventano una box.

- **σ (sigmoide) sul centro:** $\sigma(t)$ è sempre tra 0 e 1, quindi il centro predetto non esce mai dalla cella responsabile. Senza, nei primi passi di training le box salterebbero ovunque.
- **Esponenziale sulle dimensioni:** e^t è sempre positivo (niente box di larghezza negativa) e lavora in scala moltiplicativa: $t = 0$ lascia l'anchor com'è, $t = \log 2$ lo raddoppia.

In training gli obiettivi si ottengono invertendo le formule: l'offset del centro dentro la cella e il logaritmo del rapporto tra dimensione vera e dimensione dell'anchor.

$$\hat{t}_x = g_x - c_x, \quad \hat{t}_w = \log \frac{g_w}{a_w}$$

7. L'architettura



La rete del progetto con la configurazione di partenza (stride 8).

- **Backbone:** ResNet18 pre-addestrata, **troncata dopo layer2** per avere stride 8 e quindi una griglia fine 52x52. Si scartano layer3 e layer4.
- **Collo:** due blocchi conv 3x3 + batch norm + ReLU a risoluzione costante, per recuperare parte della profondità persa con il troncamento.
- **Testa:** una convoluzione 1x1 che produce i 65 numeri per posizione. È l'equivalente convoluzionale di uno strato denso applicato a ogni cella (la sliding window convoluzionale delle slide).
- La rete è **completamente convoluzionale**: funziona con qualunque dimensione multipla di 32.

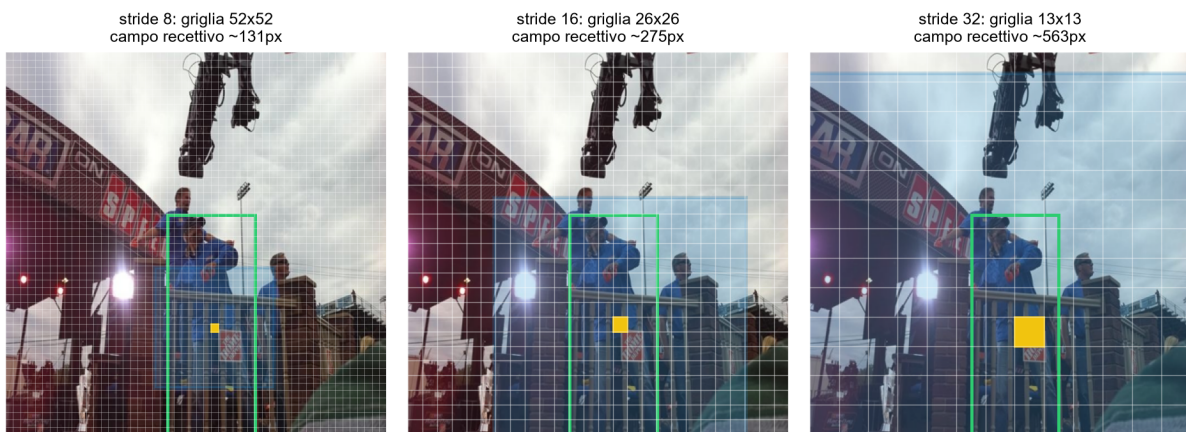
- In alternativa si può usare un backbone scritto interamente da zero (`PRETRAINED_BACKBONE = False`); la parte YOLO resta identica.

Nota

Questo capitolo descrive la **configurazione di partenza** (stride 8, ResNet18), quella da cui sono partiti gli esperimenti. La configurazione finale, scelta misurando, usa stride 16 e ResNet34: si veda il capitolo 13.

7.1 Il compromesso dello stride

Stride	Griglia	Box candidate	Parametri	Campo recettivo	Oggetti sotto una cella
8	52×52	13.520	1,59 M	~131 px	8,8%
16	26×26	3.380	3,98 M	~275 px	25,8%
32	13×13	845	15,93 M	~563 px	50,9%



Campo recettivo teorico (azzurro) della cella gialla, per i tre stride, rispetto alla persona in verde.

Lo stride regola un compromesso. Stride piccolo: griglia fine, adatta agli oggetti piccoli, ma pochi strati e campo recettivo ridotto (~131 px su 416): un oggetto grande non “entra” nella zona vista da una cella. Stride grande: la ResNet completa vede quasi tutta l'immagine (~563 px) con feature più astratte, ma una cella di 32 pixel non basta a separare gli oggetti medi. In una valutazione preliminare con stride 8 il recall sugli oggetti grandi è risultato **più basso** di quello sui medi, e le classi più grandi (Table, Cabinet) hanno avuto l'AP più bassa: è il sintomo di un campo recettivo insufficiente, che gli esperimenti e2 ed e3 verificano.

8. La loss

8.1 Costruzione dei target

- Per ogni oggetto normale: cella del centro, anchor di forma più simile, obiettivi di coordinate e classe su quella sola coppia cella-anchor, objectness = 1.
- Per ogni **oggetto sotto soglia** (lato minore sotto una cella): nessun obiettivo, e la cella del centro è **ignorata** per tutti gli anchor.
- Per ogni **box crowd**: nessun obiettivo, e sono ignorate **tutte le celle** il cui centro cade dentro la box.

- Tutte le altre coppie cella-anchor sono “vuote”: objectness = 0.

8.2 I quattro termini

Termine	Funzione	Dove si applica	Peso
Coordinate (x, y)	MSE tra $\sigma(t_x)$, $\sigma(t_y)$ e l'offset vero	anchor responsabili	$\lambda_{coord} = 5$
Coordinate (w, h)	MSE tra t_w , t_h e $\log(g/a)$	anchor responsabili	$\lambda_{coord} = 5$
Objectness	BCE con obiettivo 1	anchor responsabili	1
No-object	BCE con obiettivo 0	anchor vuoti e non ignorati	$\lambda_{noobj} = 10$
Classe	cross-entropy sulle 8 classi	anchor responsabili	1

$$\mathcal{L} = \frac{\lambda_{coord} (\mathcal{L}_{xy} + \mathcal{L}_{wh}) + \mathcal{L}_{obj} + \mathcal{L}_{cls}}{N_{obj}} + \lambda_{noobj} \frac{\mathcal{L}_{noobj}}{N_{vuote}}$$

- **Perché $\lambda_{coord} = 5$:** le coordinate sono la parte difficile del compito e i loro errori numerici sono piccoli; il peso le mette sullo stesso piano degli altri termini.
- **Lo sbilanciamento:** su 13.520 coppie cella-anchor, un'immagine con 8 oggetti ne ha 8 responsabili e più di 13.000 vuote. Se il termine no-object fosse una somma, dominerebbe: la rete imparerebbe che la risposta migliore è “non c'è mai nulla”.
- **Media sulle celle vuote:** il progetto normalizza il no-object come media sulle celle vuote, non per numero di oggetti. Così il suo peso non cambia quando cambia la griglia (52×52 ha 16 volte le celle di 13×13).
- **Perché $\lambda_{noobj} = 10$:** nel YOLO originale (98 predizioni per immagine, circa 4 oggetti, somma con peso 0,5) il peso complessivo del “non c'è nulla” è circa 12 volte quello degli oggetti. Con la media, un λ intorno a 10 riproduce quel rapporto. Con $\lambda = 0,5$ la penalità sui falsi positivi sarebbe circa 24 volte più debole: la rete vedrebbe oggetti ovunque e la precision crollerebbe.

Un limite noto

Se due oggetti hanno il centro nella stessa cella e scelgono lo stesso anchor, l'ultimo sovrascrive il primo: uno dei due non ha obiettivo. Con una griglia 52×52 e più anchor è raro, ma con griglie grossolane cresce.

9. Inferenza: IoU e non-max suppression

9.1 Dalle uscite alle box

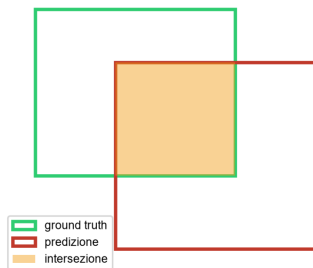
$$\text{score} = \sigma(t_o) \cdot \max_c \text{softmax}(t_{cls})_c$$

- Si decodificano tutte le box candidate e si calcola lo **score**: probabilità che ci sia un oggetto per probabilità della classe più probabile.
- Si scartano le box con score sotto la **soglia di confidenza**; per sicurezza se ne tengono al massimo 300 per immagine.
- Si applica la **NMS per classe**: due box di classi diverse possono sovrapporsi legittimamente (una persona davanti a un'auto).

9.2 Intersection over Union

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

IoU = intersezione / unione = 0.24



Esempio

Area verde 7,50, area rossa 8,40, intersezione 3,06.

$\text{IoU} = 3,06 / (7,50 + 8,40 - 3,06) = \mathbf{0,238}$.

Vale 1 per box identiche, 0 per box disgiunte. È simmetrica e non dipende dalla scala.

9.3 Non-max suppression

Più celle e anchor vicini tendono a predire lo stesso oggetto. L'algoritmo delle slide:

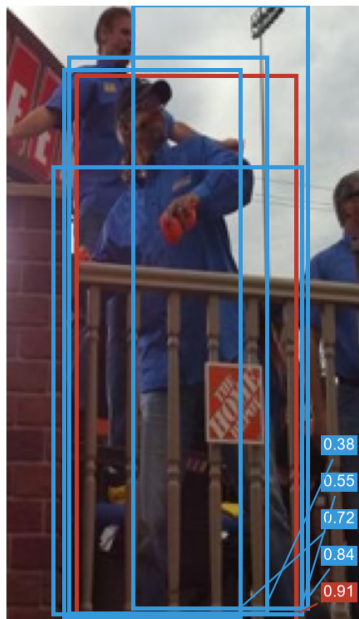
ordina le box per score decrescente

finché restano box:

prendi quella con score massimo -> è una predizione finale

scarta tutte le altre con $\text{IoU} > \text{soglia}$ (0.45) rispetto a lei

prima: 5 box sopra la soglia



dopo l'NMS ($\text{IoU} > 0,45$ scartate): 1 box



Cinque box sopra la soglia sullo stesso oggetto: tutte hanno IoU tra 0,63 e 0,80 con la migliore, quindi l'NMS tiene solo quella con score 0,91.

10. Come si misura un detector

10.1 Predizioni corrette e sbagliate

- Una predizione è un **vero positivo** (TP) se ha IoU $\geq 0,5$ con un oggetto vero della stessa classe non ancora abbinato. Le predizioni si considerano in ordine di score: le più sicure scelgono per prime.
- Altrimenti è un **falso positivo** (FP): box sbagliata, classe sbagliata o duplicato di un oggetto già trovato.
- Gli oggetti mai abbinati sono **falsi negativi**.
- **Regioni ignorate** (regole di COCO): non contano una predizione abbinata a un oggetto sotto soglia, una predizione contenuta per almeno metà in una box crowd della sua classe, e una predizione non abbinata a sua volta sotto soglia.

$$P = \frac{TP}{TP + FP} \quad R = \frac{TP}{N_{oggetti}} \quad F_1 = \frac{2PR}{P + R}$$

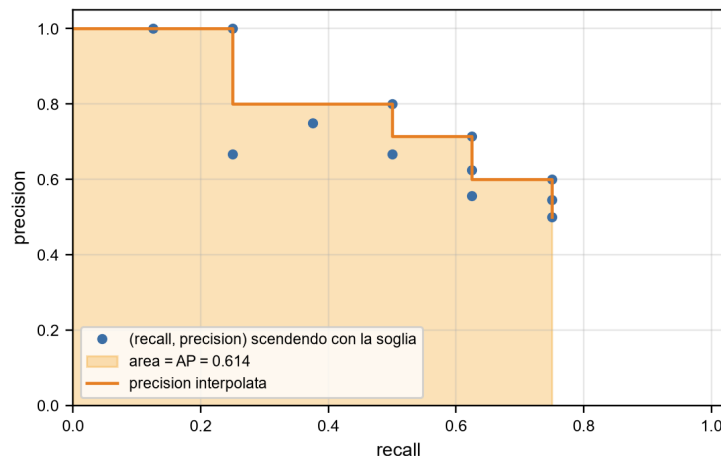
10.2 La trappola della soglia fissa

Precision e recall cambiano con la soglia di confidenza: alzandola si scartano box, la precision sale e il recall scende. Confrontare due modelli a una soglia fissa è ingannevole, perché modelli diversi sono calibrati in modo diverso. In una valutazione preliminare lo stesso modello aveva **F1 = 0,145** alla soglia 0,25 e **F1 = 0,249** alla soglia 0,65: confrontato a 0,25 sarebbe sembrato molto peggiore di quanto era.

10.3 Average Precision e mAP

Si raccolgono tutte le predizioni (da confidenza 0,01 in su), si ordinano per score e si scende lungo la lista: a ogni passo si ricalcolano precision e recall. Si ottiene una curva precision-recall; la precision viene **interpolata** (a ogni recall si prende la precision massima ottenibile a recall uguale o maggiore) e l'area sotto la curva è l'**AP** della classe. La **mAP** è la media sulle classi.

$$AP = \sum_k (R_k - R_{k-1}) P_k^{interp}, \quad mAP = \frac{1}{C} \sum_{c=1}^C AP_c$$



Esempio: 12 predizioni ordinate per score su 8 oggetti (sequenza TP/FP: TP TP FP TP TP FP TP FP FP TP FP FP). AP = 0,614; il recall si ferma a 0,75 perché 2 oggetti non sono mai trovati.

- La **mAP@0.5 non dipende dalla soglia**: è la metrica con cui si scelgono il checkpoint e la variante migliore.
- Per usare il detector serve comunque una soglia: si sceglie quella che massimizza l'F1 sul validation, e viene salvata nel checkpoint e usata da `detect.py`.
- Il report divide il recall per fascia di dimensione (medi: lato minore tra 1/20 e 3/20; grandi: oltre) per capire dove il modello fatica.

11. Il training

Aspetto	Scelta	Motivo
Pre-elaborazione	ridimensionamento a 416×416, normalizzazione ImageNet	stesse statistiche con cui ResNet18 è stata pre-addestrata
Augmentation	flip orizzontale ($p = 0,5$), luminosità e contrasto, zoom e traslazione $\pm 30\%$	più varietà senza nuove etichette; il flip ribalta anche le box ($c_x \rightarrow 1 - c_x$)
Ottimizzatore	Adam, learning rate 10^{-4} con decadimento coseno, weight decay $5 \cdot 10^{-4}$	learning rate basso per non rovinare il backbone pre-addestrato
Batch / epoche	16 / 30	circa 720 passi per epoca
Selezione	mAP@0.5 sul validation a ogni epoca	la loss può scendere senza che le detection migliorino
Riproducibilità	configurazione salvata nel checkpoint e verificata al caricamento	un checkpoint caricato con parametri diversi darebbe risultati sbagliati in silenzio

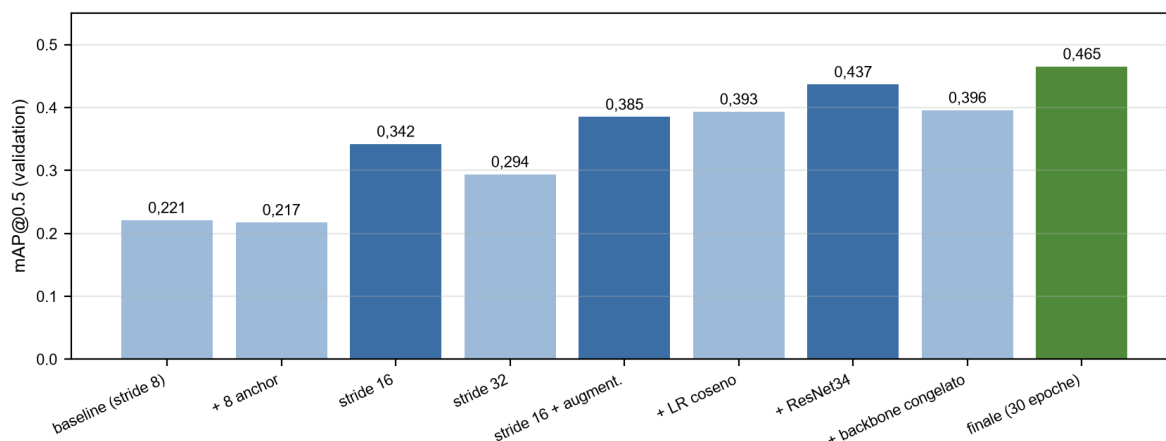
L'augmentation geometrica opzionale (esperimento e4) ingrandisce o rimpicciolisce l'immagine fino al 30% e la sposta, trasformando le box di conseguenza. Una box che dopo il taglio resta visibile per meno del 40% diventa una regione ignorata, per lo stesso motivo delle box crowd.

Per gli esperimenti ogni variante si lancia senza modificare il codice, passando i parametri in una variabile d'ambiente. Il codice viene copiato nella cartella dell'esperimento, che salva checkpoint e storico delle metriche per epoca: ogni risultato resta riproducibile.

12. Gli esperimenti di ottimizzazione

Invece di modificare molte cose insieme, ogni esperimento cambia **una sola** scelta rispetto alla baseline (*ablation*). Così l'effetto di ogni scelta è misurato separatamente e si può motivare. Alla fine si combinano le modifiche che migliorano la mAP@0.5 sul validation.

Esperimento	Base	Cosa cambia	mAP@0.5 val.
e0_baseline	baseline	nessuna: configurazione di partenza	0,221
e1_anchor8	baseline	8 anchor: 4 scale × 2 rapporti	0,217
e2_stride16	baseline	stride 16 (griglia 26×26)	0,342
e3_stride32	baseline	stride 32 (griglia 13×13)	0,294
e4_jitter_s16	stride 16	zoom e traslazione casuali (±30%)	0,385
e5_cosine_s16j	e4	learning rate con decadimento coseno	0,393
e6_resnet34_s16j	e4	backbone ResNet34	0,437
e7_freeze3_s16j	e4	backbone congelato per 3 epoche	0,396
finale	e4	ResNet34 + coseno + 30 epoche	0,465



mAP@0.5 sul validation. Le barre blu scuro sono le modifiche adottate; e5, e6 ed e7 partono tutte da e4, non si sommano fra loro.

12.1 Cosa ha funzionato e cosa no

- **Stride 16: +0,121** (0,221 → 0,342), il salto più grande. Conferma la diagnosi del campo recettivo: con stride 8 le classi grandi (Table 0,03, Cabinet 0,08) erano quasi invisibili, con stride 16 salgono a 0,13 e 0,20. Stride 32 peggiora (0,294): il campo recettivo è ancora più ampio, ma celle da 32 px non bastano a separare gli oggetti medi, che sono la maggioranza.
- **Augmentation: +0,043** (0,342 → 0,385) ed elimina l'overfitting: senza, la loss sul validation risaliva da 2,88 a 3,75 nelle ultime epoche; con, resta al minimo fino alla fine.
- **ResNet34: +0,052** (0,385 → 0,437), il secondo contributo. Un backbone più profondo migliora tutte le classi, in particolare quelle difficili.
- **Learning rate coseno: +0,008**, dentro il rumore come valore, ma la curva finale è più stabile: da 0,376/0,385/0,376 a una salita regolare. Adottato per questo motivo, non per il numero.
- **8 anchor: -0,004**. Gli anchor coprono meglio le forme (box mal coperte dal 33% al 13%), ma il risultato non cambia: **non erano il collo di bottiglia**. Scartato.

- **Backbone congelato per 3 epoche: +0,011** sul massimo, ma l'ultima epoca è più bassa della variante senza congelamento (0,379 contro 0,383): il massimo è un picco isolato. Scartato.

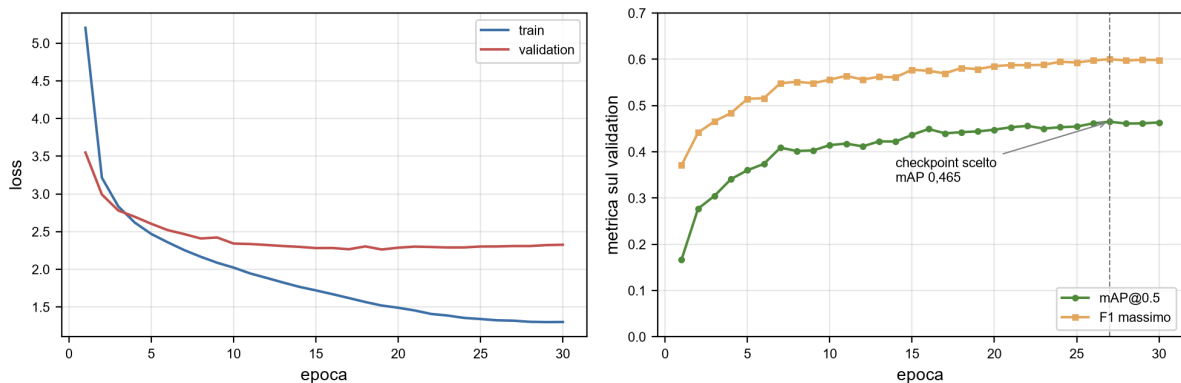
Attenzione al rumore

Ogni variante è addestrata una volta sola: differenze di mAP molto piccole (per esempio sotto 0,005) possono dipendere dal caso (ordine dei dati, inizializzazione della testa) più che dalla modifica. Si considerano significativi solo miglioramenti netti.

13. Risultati

13.1 La configurazione finale

Scelta	Valore	Perché
Stride	16 (griglia 26×26)	il salto più grande: +0,121 di mAP
Augmentation	zoom e traslazione ±30%	+0,043 e nessun overfitting
Backbone	ResNet34 pre-addestrata	+0,052
Learning rate	10^{-4} con decadimento coseno	curva finale più stabile
Epoche	30	a 20 epoche il mAP saliva ancora
Anchor	i 5 originali	gli 8 anchor non hanno migliorato nulla



Run finale. A sinistra le loss: quella di validation smette di scendere ma non risale, quindi nessun overfitting. A destra le metriche sul validation; il checkpoint scelto è l'epoca 27.

Il training è durato circa 64 minuti (128 secondi per epoca su una RTX 4070 Ti). Il checkpoint migliore è quello dell'epoca 27, con **mAP@0.5 = 0,465** sul validation e F1 massimo 0,600 alla soglia di confidenza 0,90.

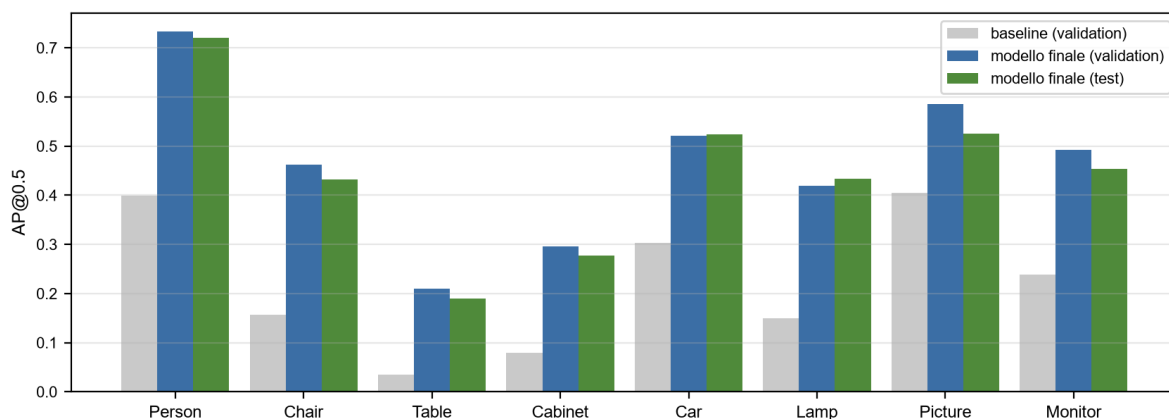
13.2 Il risultato sul test set

Il modello scelto è stato valutato **una sola volta** sulle 1.499 immagini del test (8.383 oggetti del compito), con la soglia di confidenza scelta sul validation.

Metrica	Validation	Test
mAP@0.5	0,465	0,444
F1 (a confidenza > 0,9)	0,600	0,588
Precision	—	0,660
Recall	—	0,531

Il calo tra validation e test è piccolo e atteso: sul validation sono stati scelti checkpoint, soglia e configurazione, quindi quel numero è leggermente ottimistico. **Il risultato da riportare è quello sul test.**

Classe	Precision	Recall	F1	AP@0.5	Oggetti
Person	0,745	0,703	0,724	0,720	4.115
Chair	0,618	0,388	0,477	0,432	1.419
Table	0,358	0,183	0,242	0,190	687
Cabinet	0,466	0,275	0,345	0,277	761
Car	0,639	0,524	0,576	0,524	487
Lamp	0,494	0,463	0,478	0,433	469
Picture	0,609	0,465	0,528	0,525	288
Monitor	0,583	0,401	0,475	0,453	157
totale	0,660	0,531	0,588	0,444	8.383

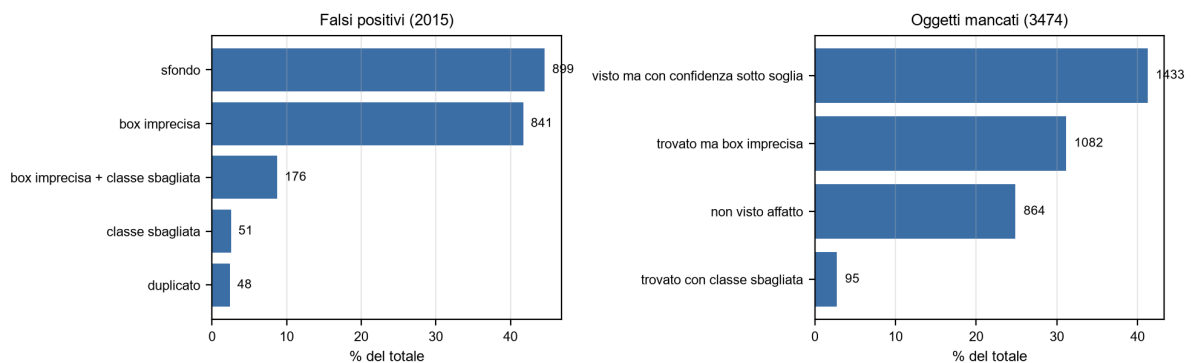


AP per classe: baseline, modello finale sul validation e modello finale sul test.

- **Person** è la classe migliore (AP 0,72): è anche la più frequente, con quasi la metà degli oggetti.
- **Table** resta la più difficile (0,19): i tavoli sono spesso coperti da oggetti, tagliati dal bordo e con confini ambigui.
- **Il campo recettivo non è più un problema**: sul test il recall sugli oggetti grandi (0,58) supera quello sui medi (0,51), mentre nella baseline era il contrario (0,38 contro 0,56).

13.3 Dove sbaglia il modello

Analizzando le predizioni sul validation alla soglia d'uso si vede che il problema principale non è riconoscere l'oggetto, ma delimitarlo.



Scomposizione degli errori del modello finale (validation, soglia 0,9).

- **Box imprecise:** 42% dei falsi positivi e 31% degli oggetti mancati. Il modello trova l'oggetto ma la box non arriva a IoU 0,5, quindi lo stesso errore conta due volte.
- **Confidenza poco separata:** il 41% degli oggetti mancati era stato predetto correttamente, ma sotto la soglia. Alzare la soglia perde oggetti, abbassarla aggiunge falsi positivi.
- **Classificazione corretta:** solo il 2,5% dei falsi positivi è un errore di classe. Le confusioni sono quasi tutte fra Chair e Table.
- **Etichette mancanti:** controllando a occhio le predizioni “sullo sfondo” più sicure, molte sono oggetti reali non annotati (un'auto, un mobile, alcune sedie). Una parte di questi falsi positivi non è un errore del modello, e il valore reale è quindi un po' più alto di quello misurato.

13.4 Il riconoscitore su una foto

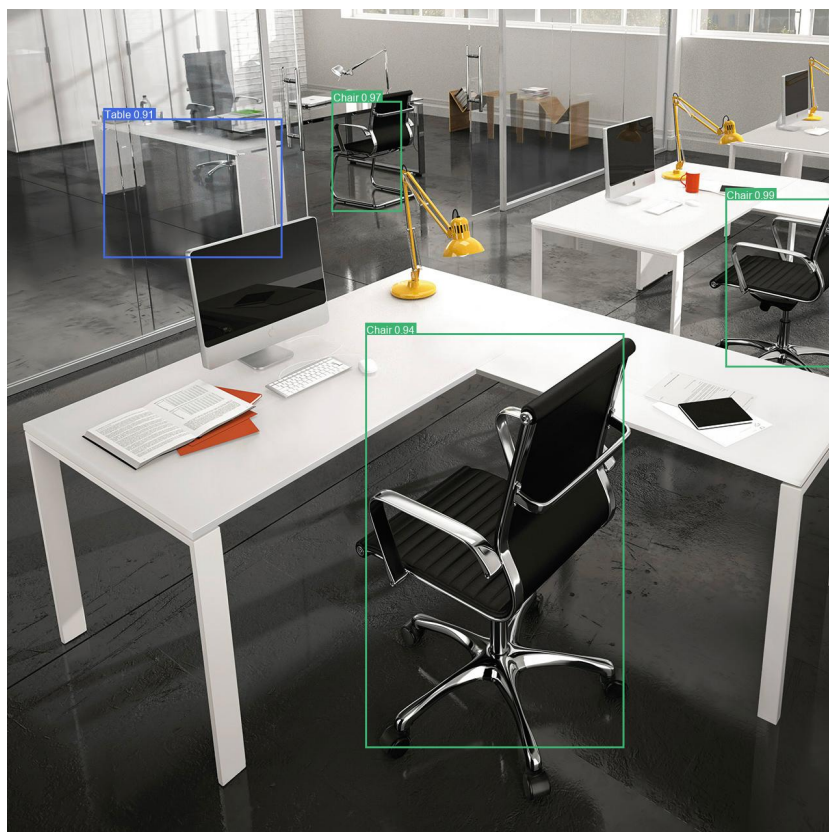
La consegna chiede un riconoscitore per una foto scattata con una fotocamera. Questa è una foto di prova, mai vista dal modello, elaborata con `detect.py` alla soglia salvata nel checkpoint.



Detection sulla foto: entrambe le persone trovate con confidenza 0,96 e 0,98, nessun falso positivo. Le due figure parziali sul bordo sinistro, piccole e coperte dai fiori, restano sotto la soglia.

13.5 Un secondo esempio: una scena d'ufficio

La seconda immagine (una scena d'ufficio con più classi insieme) mostra bene sia i pregi sia i limiti. Il modello trova **tre sedie su quattro**, con box precise, e un tavolo sullo sfondo; tutte le predizioni sono corrette, quindi la precision è perfetta.



Quattro oggetti rilevati, tutti corretti. Restano fuori le due scrivanie in primo piano, i monitor e le lampade.

- **Le scrivanie in primo piano non vengono trovate**, pur essendo gli oggetti più grandi: è la debolezza misurata sul test (Table ha AP 0,19, la più bassa). Qui sono anche tagliate dal bordo, a forma di L e coperte da oggetti.
- **Monitor e lampade restano sotto soglia**: uno schermo nero spento su sfondo chiaro e una lampada snodata sono lontani dagli esempi tipici del dataset.
- **Precision alta, recall basso**: è il comportamento atteso alla soglia 0,9, scelta per privilegiare la correttezza delle box rispetto al numero di oggetti trovati. Abbassando la soglia si trovano più oggetti, ma arrivano anche i falsi positivi.

Il risultato in una frase

Su foto mai viste il detector trova circa la metà degli oggetti richiesti (0,53 di recall) e due box su tre sono corrette (0,66 di precision). Funziona bene sulle persone, discretamente su auto, quadri e monitor, male su tavoli e mobili.

14. Come è organizzato il codice

File	Cosa fa
config.py	Classi, geometria, anchor, soglie, iperparametri, override per gli esperimenti.
prepare_data.py	Scarica gli shard, filtra le classi, marca le box crowd, divide in train/val/test.
analyze_sizes.py	Statistiche sulle dimensioni: griglia, soglie, anchor.
dataset.py	Dataset PyTorch: lettura di immagini ed etichette, augmentation.
model.py	Backbone (ResNet18/34 o da zero), collo e testa.
loss.py	Costruzione dei target, regioni ignorate, loss YOLO, decodifica.
utils.py	IoU, conversioni tra formati di box, NMS, tutto scritto a mano.
train.py	Training, valutazione a ogni epoca, checkpoint e storico.
evaluate.py	Abbinamento predizioni-oggetti, precision/recall/F1, AP e mAP.
detect.py	Detection su una foto e disegno delle box.

```
python src/prepare_data.py          # dati (15 shard)
python src/train.py                 # training
python src/evaluate.py --split test  # valutazione finale
python src/detect.py foto.jpg --out risultato.jpg
```

```
# una variante, senza modificare il codice (PowerShell)
$env:ES1_CONFIG = '{"EXPERIMENT": "e2_stride16", "STRIDE": 16}'
python src/train.py
```

15. Limiti e possibili miglioramenti

Limite	Possibile intervento
Una sola scala di predizione: compromesso tra oggetti piccoli e grandi	Predizioni su più scale (feature pyramid, come YOLOv3)
Ridimensionamento a quadrato: le proporzioni vengono deformate	Letterbox: ridimensionare mantenendo le proporzioni e riempire i bordi
Un oggetto per coppia cella-anchor	Assegnare un oggetto a più anchor, o griglie più fini
Etichette rumorose (circa un ritaglio su dieci sbagliato)	Pulizia delle etichette, o loss più robuste
Classi sbilanciate (Person oltre il 50%)	Campionamento bilanciato o pesi per classe nella loss
Poche immagini rispetto a Object365 completo	Più shard: è la leva più forte, a costo di training più lunghi
Valutazione solo a IoU 0,5	mAP media su IoU da 0,5 a 0,95 (metrica COCO), più severa sulla precisione delle box

16. Domande d'esame probabili

1. Perché YOLO e non una sliding window?

La sliding window classifica migliaia di ritagli separatamente. YOLO predice tutte le box in un solo passaggio convoluzionale sull'immagine intera, condividendo i calcoli: è molto più veloce.

2. A quale cella viene assegnato un oggetto?

A quella che contiene il suo centro. Solo quella cella, con l'anchor di forma più simile, ha l'obiettivo di predirlo.

3. A cosa servono gli anchor?

Permettono a una cella di predire più oggetti e fanno specializzare le predizioni su forme tipiche (alte, larghe, piccole, grandi). La rete predice solo una correzione dell'anchor.

4. Come si sceglie l'anchor responsabile?

Con l'IoU tra le sole forme (larghezza e altezza, box centrate nello stesso punto): vince l'anchor con IoU massima.

5. Perché la sigmoide sul centro e l'esponenziale sulle dimensioni?

La sigmoide tiene il centro dentro la cella responsabile e stabilizza il training; l'esponenziale garantisce dimensioni positive e scala l'anchor in modo moltiplicativo.

6. Cosa succede se il peso del no-object è troppo basso? E troppo alto?

Troppo basso: la rete predice oggetti ovunque e la precision crolla. Troppo alto: impara che non c'è mai nulla e il recall crolla. Il progetto usa la media sulle celle vuote con $\lambda = 10$.

7. Calcola l'IoU di due box.

$\text{IoU} = \text{intersezione} / (\text{area A} + \text{area B} - \text{intersezione})$. Nell'esempio: $3,06 / (7,50 + 8,40 - 3,06) = 0,238$.

8. Come funziona la NMS e perché per classe?

Si tiene la box con score massimo e si scartano quelle che la sovrappongono con IoU sopra soglia, poi si ripete. Per classe, perché oggetti di classi diverse possono sovrapporsi davvero.

9. Cos'è il campo recettivo e perché conta qui?

La porzione di immagine che influenza un'unità. Con stride 8 e rete troncata è ~131 px su 416: gli oggetti grandi non sono visti per intero da una cella, e infatti sono riconosciuti peggio.

10. Perché una ResNet pre-addestrata?

Transfer learning: i filtri appresi su ImageNet riconoscono già bordi, texture e parti di oggetti. Con circa 11.000 immagini di training partire da pesi casuali darebbe risultati molto peggiori.

11. Cosa sono le box crowd e come le tratti?

Box attorno a gruppi di oggetti non annotati singolarmente. Sono regioni ignorate: niente obiettivi, niente penalità no-object sulle celle coperte, e in valutazione le predizioni al loro interno non contano.

12. Perché non cancellare semplicemente gli oggetti piccoli?

Resterebbero nell'immagine: la loss insegnerebbe che in quel punto non c'è nulla, penalizzando anche oggetti simili più grandi. Ignorarli evita il segnale contraddittorio.

13. Perché la mAP e non l'F1?

L'F1 dipende dalla soglia di confidenza e modelli diversi sono calibrati diversamente. La mAP integra tutte le soglie. L'F1 massimo serve poi per scegliere la soglia d'uso.

14. Come si calcola l'AP?

Si ordinano le predizioni per score, si calcolano precision e recall cumulati, si interpola la precision (massimo a recall maggiore o uguale) e si somma l'area: $\sum (R_k - R_{k-1}) \cdot P_k$.

15. Perché validation e test separati?

Il validation si usa per scegliere checkpoint e varianti, quindi il suo punteggio è ottimistico. Il test non influenza nessuna scelta e dà una stima onesta.

16. Perché un esperimento alla volta?

Per attribuire ogni miglioramento alla sua causa. Cambiando più cose insieme non si sa quale ha aiutato e quale ha peggiorato.

17. Perché le categorie simili sono unite?

Un SUV etichettato come sfondo accanto a un'auto etichettata come oggetto dà segnali contraddittori. Unirle rende coerente la supervisione e aumenta gli esempi.

18. Cosa miglioreresti?

Predizioni multi-scala, letterbox al posto della deformazione, più dati, bilanciamento delle classi, valutazione anche a IoU più severa.

17. Glossario

Termine	Significato
Anchor box	Forma di box predefinita associata a ogni cella; la rete ne predice una correzione.
AP / mAP	Area sotto la curva precision-recall di una classe / media sulle classi.
Backbone	Parte della rete che estrae le feature (qui ResNet18).
Box crowd	Box attorno a un gruppo di oggetti non annotati singolarmente; regione ignorata.
Campo recettivo	Porzione di immagine che influenza un'unità della rete.
Cella responsabile	Cella che contiene il centro di un oggetto e ha il compito di predirlo.
Collo (neck)	Strati tra backbone e testa.
IoU	Intersection over Union: area comune diviso area unione di due box.
Logit	Valore grezzo prima di sigmoide o softmax.
NMS	Non-max suppression: elimina le box duplicate sullo stesso oggetto.
Objectness	Probabilità predetta che una coppia cella-anchor contenga un oggetto.
Regione ignorata	Zona che non produce né obiettivi né penalità.
Stride	Fattore di riduzione della risoluzione tra immagine e feature map.
Testa (head)	Strato finale che produce le predizioni per ogni cella e anchor.
Transfer learning	Riutilizzo di una rete pre-addestrata su un altro compito.